

# ICL-Traffic Forecasting: Lightweight In-Context Learning for Pretrained Spatio-Temporal Foundation Models

Xuesi Zhou

*Department of Computer Science  
North Carolina State University  
Raleigh, NC, USA  
xzhou38@ncsu.edu*

Mrudani Vishal Hada

*Department of Computer Science  
North Carolina State University  
Raleigh, NC, USA  
mhada2@ncsu.edu*

Parth Ripal Parikh

*Department of Computer Science  
North Carolina State University  
Raleigh, NC, USA  
pparikh2@ncsu.edu*

**Abstract**—Pretrained spatio-temporal foundation models such as OpenCity give strong zero-shot traffic forecasts, but accuracy drops under cross-city shifts. In NLP and time-series, in-context learning (ICL) adapts predictions at inference time from retrieved demonstrations, yet ICL has not been studied for spatio-temporal foundation models. The standard recipe of feeding demonstrations into the model’s context window cannot be applied here: these models are pretrained on fixed-length sequences, and extending them introduces out-of-distribution attention patterns that degrade MAE by 9–28%. We propose ICL-Traffic, a lightweight ICL module that keeps the base model completely frozen. A 198K-parameter cross-attention aggregator is trained offline in seconds from cached features. At inference time, KNN retrieves the most similar training windows, and the aggregator combines their prediction errors into a per-query correction, with no test-time gradients. On PEMS07M, ICL-Traffic reduces MAE from 4.50 to 4.15 (+7.8%) with only ~0.8% extra parameters. On two out-of-distribution cross-city datasets (SZ-DIDI and CD-DIDI), the same module recovers 78–84% of the MAE gain that prediction-head fine-tuning achieves, without modifying a single backbone weight.

**Index Terms**—traffic forecasting, in-context learning, foundation models, spatio-temporal prediction, demonstration-based adaptation, retrieval-augmented inference

## I. INTRODUCTION

Accurate traffic forecasting supports urban applications such as route planning and signal control. Traditional spatio-temporal models such as STGCN [1] and Graph WaveNet [2] achieve strong supervised performance but require per-dataset training with target-domain labels. OpenCity [3] represents a new paradigm: a spatio-temporal foundation model (ST-FM) pretrained on large-scale heterogeneous traffic data. It integrates a Transformer backbone with graph neural networks and transfers zero-shot across cities, data categories, and time horizons. Zero-shot accuracy is strong, but it degrades once the target city differs substantially from pretraining. On SZ-DIDI and CD-DIDI, OpenCity’s zero-shot MAE significantly underperforms full-shot supervised baselines, showing the limits of static zero-shot inference. OpenCity addresses this with 3-epoch prediction-head fine-tuning on target-domain data, but this requires gradient updates. A natural question

arises: can demonstration-based adaptation at inference time match or replace fine-tuning for ST foundation models?

In NLP, in-context learning (ICL) lets foundation models adapt by conditioning on demonstrations without gradient updates [4]. Chen et al. [5] formalize ICL as a meta-learning objective through In-Context Tuning (ICT), training language models to predict target labels given concatenated instruction, in-context examples, and query. ICT unifies prompting, fine-tuning, and meta-learning into a single framework, outperforming gradient-based meta-learning such as MAML [6] by leveraging the language model’s pattern-matching inductive bias. Das et al. [7] extend ICL to time-series foundation models through In-Context Fine-Tuning (ICF), adapting TimesFM [8] to use related time series in its context window via separator tokens and cross-example attention. TimesFM-ICF achieves up to 7% improvement over the base model on the Monash benchmark and rivals per-dataset fine-tuning. These results confirm that retrieved examples improve adaptation in both language and time-series forecasting, yet ICL remains unexplored for spatio-temporal foundation models.

Both ICT and TimesFM-ICF enable ICL by modifying the model through meta-training or continued pretraining. Neither can be directly applied to ST-FMs: both assume 1-D causal sequences and require training corpora far larger than any available multi-city ST dataset (Section II-C). Moreover, simply concatenating demonstrations into the input also fails. OpenCity is pretrained on fixed-length sequences (24 patches = 288 timesteps), so extending the input puts the attention pattern out of distribution, breaks positional encodings, and disrupts cross-attention, degrading MAE by 9–28% (Section IV-F). We therefore keep the backbone frozen and the input unchanged, processing query and demonstrations independently and combining them only in output space.

We propose **ICL-Traffic**, a residual correction framework for enabling demonstration-based adaptation in frozen ST-FMs (Fig. 1). Each demonstration’s prediction error estimates the model’s bias on similar inputs, and a learned aggregation network combines these errors into a query-specific correction. Two factors drive improvement: (1) KNN retrieval selects

relevant demonstrations from a training-set pool, and (2) a learned aggregator (8K–198K parameters) replaces naive averaging with query-aware weighting. The base model stays fully frozen, and the aggregator trains offline in seconds from cached features. Our contributions are:

- 1) We are the first to bring in-context learning to spatio-temporal foundation models. Inspired by ICT [5] and TimesFM-ICF [7], we propose a residual correction framework that enables demonstration-based adaptation for frozen ST-FMs without architectural modification or base model retraining.
- 2) We systematically study two axes of the framework: demonstration retrieval (random vs. raw-L2 KNN vs. embedding-space KNN) and correction aggregation (naive average vs. cosine similarity vs. cross-attention), showing both are critical and jointly necessary to surpass zero-shot performance.
- 3) On PEMS07M, our best configuration improves MAE by +7.8% ( $4.50 \rightarrow 4.15$ ) with only 198K external parameters ( $\approx 0.8\%$  of the base model). On the two out-of-distribution datasets SZ-DIDI and CD-DIDI, where cross-city shift exposes OpenCity’s systematic bias, the same module recovers  $\sim 80\%$  of the MAE gain of gradient-based prediction-head fine-tuning, without modifying any model weights. The remaining  $\sim 20\%$  quantifies the gap between output-space correction and representation adaptation; to our knowledge this is the first such decomposition for ST-FMs.

## II. RELATED WORK

### A. Spatio-Temporal Foundation Models

Classical spatio-temporal models, including STGCN [1], Graph WaveNet [2], AGCRN [9], and ASTGCN [10], learn dataset-specific parameters and do not transfer across cities. Recent foundation models address this through large-scale pretraining across multiple cities and data types.

OpenCity [3], our base model, integrates Transformer architecture with graph neural networks. It employs instance normalization for distribution shift handling, patch embedding for long-horizon prediction, and temporal-categorical joint attention. Pretrained on heterogeneous multi-city traffic data covering flow, speed, and demand categories, OpenCity achieves zero-shot performance competitive with full-shot baselines across six benchmark datasets and shows promising scaling laws. When zero-shot performance is insufficient, OpenCity provides fast adaptation via 3-epoch fine-tuning of its prediction head on target-domain data. However, this remains gradient-based adaptation, and in-context learning has not been explored for OpenCity or any other ST-FM.

Other ST-FMs such as UniST [11], UrbanGPT [12], and FlashST [13] all adapt via fine-tuning or prompt-tuning; none explores ICL.

### B. In-Context Learning

GPT-3 [4] first showed that large language models can adapt to new tasks through few-shot demonstrations in the input

context, without gradient updates. Chen et al. [5] formalize this through In-Context Tuning (ICT): given concatenated instruction, labeled in-context examples, and a target query, the model is meta-trained to predict the target label across diverse NLP tasks. ICT outperforms gradient-based meta-learning (MAML [6]) and reduces sensitivity to example ordering and selection. Das et al. [7] extend ICL to time-series foundation models through In-Context Fine-Tuning (ICF): starting from TimesFM [8], they introduce separator tokens between in-context examples, enable cross-example causal attention, and remove positional encoding. Through continued pretraining, the model learns to borrow patterns from related time series in its context window. TimesFM-ICF achieves up to 7% improvement over the base model on the Monash benchmark and rivals per-dataset fine-tuning.

### C. Relationship to ICT and TimesFM-ICF

ICT and TimesFM-ICF are the closest prior work on ICL for foundation models. Both share our motivation of using retrieved demonstrations to adapt a pretrained model, but both require modifying the base model: ICT meta-trains the language model across hundreds of tasks; TimesFM-ICF continues pretraining with a new attention mask and no positional encoding. This modification-based paradigm is problematic for ST-FMs. First, both methods feed demonstrations into the model’s context window by concatenating them with the query. We show in Section IV-F that this fails for OpenCity because its fixed-length pretraining creates out-of-distribution attention patterns, degrading MAE by 9–28%. Second, both methods assume 1-D causal token sequences, whereas ST-FMs process structured spatio-temporal graphs with multi-variate node features, so the concatenation paradigm does not naturally extend to this setting. These observations motivate a fundamentally different design: rather than forcing demonstrations into the model’s context, we keep the backbone entirely frozen and combine demonstration information only in output space.

We position ICL-Traffic as the output-space surrogate of these methods: it uses the same ingredient (retrieved demonstrations) while keeping the backbone frozen. Our baselines reflect this positioning: (a) the foundation model’s zero-shot inference, (b) random-demonstration residual correction (to isolate retrieval quality), and (c) OpenCity’s 3-epoch prediction-head fine-tuning (a gradient-based counterpart at the same training budget).

### D. Test-Time Adaptation

Test-time adaptation (TTA) updates model parameters at inference time from self-supervised signals. TENT [14] minimises prediction entropy over batch-norm affine parameters; TTT [15] adds a self-supervised auxiliary head that is jointly trained at training time and updated at test time. Both require test-time backward passes and modify the base model. Our method is the opposite: no gradient updates happen at test time, and the aggregator is trained once, offline, from cached features. The two paradigms are complementary and could in principle be composed.

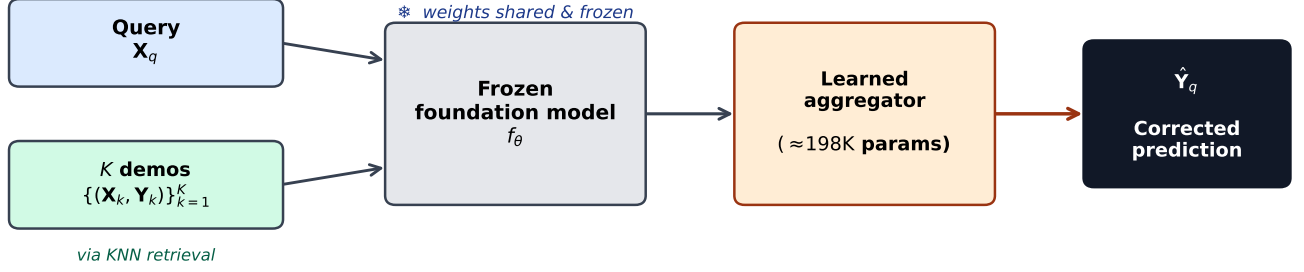


Fig. 1. ICL-Traffic pipeline: a query and  $K$  KNN-retrieved demonstrations share a frozen foundation model  $f_\theta$ ; a learned aggregator emits an output-space correction.

### III. METHOD

#### A. Problem Formulation

A traffic network has  $N$  sensor nodes. Given a window of historical observations  $\mathbf{X} \in \mathbb{R}^{T \times N \times F}$  over  $T$  time steps with  $F$  features per node, the task is to predict future values  $\mathbf{Y} \in \mathbb{R}^{T' \times N}$  over the next  $T'$  steps. Let  $f_\theta$  denote a frozen pretrained ST-FM with parameters  $\theta$ , producing  $\hat{\mathbf{Y}} = f_\theta(\mathbf{X})$ . We assume a demonstration pool  $\mathcal{D} = \{(\mathbf{X}_d, \mathbf{Y}_d)\}_{d=1}^{|\mathcal{D}|}$  drawn from the target-domain training split, where both inputs and ground-truth futures are available. For a test query  $\mathbf{X}_q$ , we want to pick  $K$  demonstrations from  $\mathcal{D}$  and return a prediction  $\hat{\mathbf{Y}}_q$  that improves on  $f_\theta(\mathbf{X}_q)$  without touching  $\theta$ .

#### B. Residual Correction Framework

We estimate and correct the frozen model's prediction error using demonstrations. For each selected demonstration  $(\mathbf{X}_k, \mathbf{Y}_k)$ , we compute the residual correction

$$\mathbf{c}_k = \mathbf{Y}_k - f_\theta(\mathbf{X}_k), \quad (1)$$

which captures the model's bias on input  $\mathbf{X}_k$ . The corrected prediction for query  $\mathbf{X}_q$  is

$$\hat{\mathbf{Y}}_q = f_\theta(\mathbf{X}_q) + \text{Aggregate}(\{\mathbf{c}_k\}_{k=1}^K, \mathbf{h}_q, \{\mathbf{h}_k\}_{k=1}^K), \quad (2)$$

where  $\mathbf{h}_q, \mathbf{h}_k$  are frozen-encoder features. Because every input is processed through the standard forward path, the input sequence length is never modified. When  $K = 0$  the framework fallbacks to zero-shot prediction. Both demo predictions and encoder features can be cached in a one-time Phase 1 pass, so aggregator training requires no additional forward passes through the base model.

#### C. Demonstration Retrieval

The quality of residual corrections depends critically on similarity between demonstrations and the query. We investigate three retrieval strategies.

*Random selection:* samples  $K$  demonstrations uniformly from  $\mathcal{D}$ . Because randomly selected demos have low correlation with the query, their corrections are effectively noise. With  $K = 3$ , random selection yields MAE 6.14, which is 36% worse than zero-shot, confirming that demo *quality* matters.

*Raw-input KNN:* retrieves the  $K$  nearest neighbors based on L2 distance in the flattened input space,

$$\mathcal{N}_K(\mathbf{X}_q) = \arg \min_{S \subset \mathcal{D}, |S|=K} \sum_{d \in S} \|\text{vec}(\mathbf{X}_q) - \text{vec}(\mathbf{X}_d)\|_2. \quad (3)$$

For PEMS07M,  $\text{vec}(\mathbf{X}) \in \mathbb{R}^{196608}$  ( $= 288 \text{ timesteps} \times 228 \text{ nodes} \times 3 \text{ features}$ ). This simple approach improves sharply over random (MAE 6.14  $\rightarrow$  4.66 for  $K = 3$ ), yet still does not surpass zero-shot. Raw L2 treats every element as an independent coordinate, so proximity reflects point-wise numerical closeness rather than shared traffic state such as congestion regime or rush-hour pattern. The retrieved residuals  $\mathbf{c}_k$  are therefore often uninformative, limiting the corrector's signal-to-noise ratio.

*Embedding-space KNN:* retrieves nearest neighbors in the frozen encoder's feature space after mean-pooling over nodes. Let  $\mathbf{h}_n \in \mathbb{R}^{512}$  be the per-node feature produced by the encoder for the window. We pool across nodes and retrieve in the pooled space,

$$\bar{\mathbf{h}} = \frac{1}{N} \sum_{n=1}^N \mathbf{h}_n \in \mathbb{R}^{512}, \quad (4)$$

$$\mathcal{N}_K(\mathbf{X}_q) = \arg \min_S \sum_{d \in S} \|\bar{\mathbf{h}}_q - \bar{\mathbf{h}}_d\|_2. \quad (5)$$

The encoder features come from the same Phase-1 cache used for corrections. Proximity in this space reflects similarity of *predictive context* rather than of raw numerical values. We mean-pool to a single 512-d vector per window to keep retrieval cheap. Under the same learned aggregator, swapping raw-L2 for embedding KNN reduces PEMS07M MAE from 4.29 to 4.15, an extra 3.3% relative gain from the retrieval step alone (Fig. 2).

*Scope of embedding KNN in this paper:* We use embedding KNN as the in-distribution upper bound on PEMS07M but keep raw-L2 KNN for SZ-DIDI and CD-DIDI. The core contribution of this paper is the residual correction framework itself, not the retrieval metric. Raw-L2 already demonstrates the framework's effectiveness on OOD cities. Whether embedding KNN further improves OOD retrieval is an orthogonal question that we leave to future work.

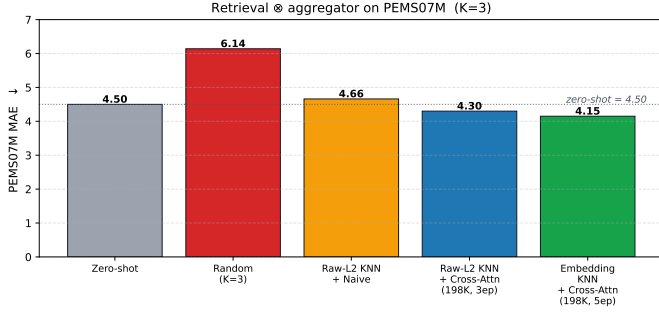


Fig. 2. PEMS07M MAE across retrieval@aggregator combinations ( $K=3$ ): zero-shot, random, raw-L2 KNN with naive averaging, raw-L2 KNN with cross-attention, and embedding KNN with cross-attention.

#### D. Cross-Attention Aggregator

Naive uniform averaging  $(1/K) \sum_k \mathbf{c}_k$  dilutes informative corrections with residual noise and fails to surpass zero-shot. We replace it with a learned aggregation module. We compare a minimal 8K-parameter cosine-similarity variant (SIMPLEDEMOAGGREGATOR) against a 198K-parameter cross-attention variant (DEMOAGGREGATOR); the latter is our main design, detailed below and illustrated in Fig. 3.

*Inputs:* The aggregator receives three tensors drawn from the cached Phase-1 outputs:

$$\begin{aligned} \mathbf{h}_q &\in \mathbb{R}^{B \times N \times D}, & (\text{query features, } D = 512) \\ \{\mathbf{h}_k\} &\in \mathbb{R}^{B \times K \times N \times D}, & (\text{demo features}) \\ \{\mathbf{c}_k\} &\in \mathbb{R}^{B \times K \times T' \times N \times 1}. & (\text{demo residuals}) \end{aligned}$$

*Per-node, per-head attention over demonstrations:* Queries  $\mathbf{Q}$  and keys  $\mathbf{K}$  are obtained by layer-normalising the encoder features and projecting them to  $H \cdot d_h$  dimensions ( $H = 4$  heads,  $d_h = 32$ , internal  $d_{\text{proj}} = 128$ ):

$$\mathbf{Q} = \mathbf{W}_Q \text{LN}(\mathbf{h}_q) \in \mathbb{R}^{B \times N \times H \times d_h}, \quad (6)$$

$$\mathbf{K} = \mathbf{W}_K \text{LN}(\mathbf{h}_k) \in \mathbb{R}^{B \times K \times N \times H \times d_h}. \quad (7)$$

For every batch element  $b$ , node  $n$ , and head  $h$ , we softmax over the  $K$  demonstrations:

$$\alpha_{b,n,h,k} = \text{softmax}_k \left( \frac{\mathbf{Q}_{b,n,h} \cdot \mathbf{K}_{b,k,n,h}^\top}{\sqrt{d_h}} \right). \quad (8)$$

This yields a *per-node* demo-weight vector per head, which is critical for spatio-temporal data where different sensors may match different demonstrations.

*Values are the raw corrections (no V-projection):* Standard cross-attention projects values through a learned  $\mathbf{W}_V$  before weighting. Here, the corrections  $\{\mathbf{c}_k\}$  already live in the prediction space  $\mathbb{R}^{T' \times N \times 1}$ , so we use them directly as values with no projection. The attention weights  $\alpha$  from Q/K determine how much each demo’s correction contributes, and the weighted sum produces a candidate correction per head:

$$\tilde{\mathbf{c}}_{b,t,n}^{(h)} = \sum_{k=1}^K \alpha_{b,n,h,k} \mathbf{c}_{b,k,t,n} \in \mathbb{R}^{B \times T' \times N}. \quad (9)$$

*Head combination:* A linear layer with no bias, initialised to  $1/H$ , merges the  $H$  per-head candidates,

$$\bar{\mathbf{c}}_{b,t,n} = \sum_{h=1}^H w_h \tilde{\mathbf{c}}_{b,t,n}^{(h)}, \quad w_h \in \mathbb{R}. \quad (10)$$

The init-to-average lets the aggregator recover uniform head-averaging at step 0, then deviate from it as training progresses.

*Query-conditioned output gate:* A per-node multiplicative gate  $g_n \in \mathbb{R}_{\geq 0}$  is computed from the query feature by a small two-layer MLP with softplus output and *zero-initialised* final layer:

$$g_n = \text{softplus}(\mathbf{W}_2 \text{SiLU}(\mathbf{W}_1 \mathbf{h}_{q,n})), \quad \mathbf{W}_2 \leftarrow \mathbf{0}. \quad (11)$$

The hidden activation is SiLU ( $x \cdot \sigma(x)$ ), also known as Swish). Zero-init makes  $g_n \approx \text{softplus}(0) \approx 0.693$  at the start of training, so the aggregator begins by lightly scaling the head-averaged correction rather than saturating or zeroing it. Softplus is chosen over sigmoid to allow *amplification* of corrections (gate  $> 1$ ), which is empirically useful under large distribution shifts (Section IV).

*Output:* The final per-query correction is

$$\Delta_q = g \cdot \bar{\mathbf{c}} \in \mathbb{R}^{B \times T' \times N \times 1}, \quad \hat{\mathbf{Y}}_q = f_\theta(\mathbf{X}_q) + \Delta_q. \quad (12)$$

*Parameter budget:* LN:  $2D = 1,024$ ;  $\mathbf{W}_Q$ :  $D \cdot 128 + 128 = 65,664$ ;  $\mathbf{W}_K$ :  $65,664$ ; head-combine: 4; gate-MLP ( $D \rightarrow 128 \rightarrow 1$ , zero-init):  $65,793$ . Total:  $\approx 198$  K, i.e.  $\approx 0.8\%$  of the 26M-parameter OpenCity-plus backbone.

#### E. Training Procedure

Training proceeds in two phases. *Phase 1* (one-time) runs the frozen base model forward over all training windows and caches, for every  $d \in \mathcal{D}$ , the prediction  $f_\theta(\mathbf{X}_d)$ , the encoder features  $\mathbf{h}_d$  (both pooled and non-pooled), and the correction  $\mathbf{c}_d$ . *Phase 2* trains the aggregator from cached data only, with no forward or backward pass through the base model. We use Adam at learning rate  $10^{-4}$ , MAE loss, and early stopping with patience 5. Phase 1 is amortised across all downstream experiments (different  $K$ , retrievers, and aggregators), making the ablations in Section IV tractable.

## IV. EXPERIMENTS

#### A. Experimental Setup

**Datasets.** We evaluate on three traffic datasets spanning two distribution regimes. *PEMS07M* contains 228 highway speed sensors in California at 5-minute intervals (50/10/40 train/val/test split, 5,761 training windows). It is not present in OpenCity’s pretraining corpus but has the same modality (highway speed) as several pretraining datasets; we use it as our *in-distribution* benchmark. *SZ-DIDI* (Shenzhen, 627 nodes) and *CD-DIDI* (Chengdu, 524 nodes) are ride-hailing demand datasets at 10-minute intervals, used by OpenCity’s paper to report fast-adaptation results. They represent the *out-of-distribution* regime: unseen cities, unseen modality (ride-hail demand vs. highway speed), and a different temporal resolution.

### Cross-attention aggregator

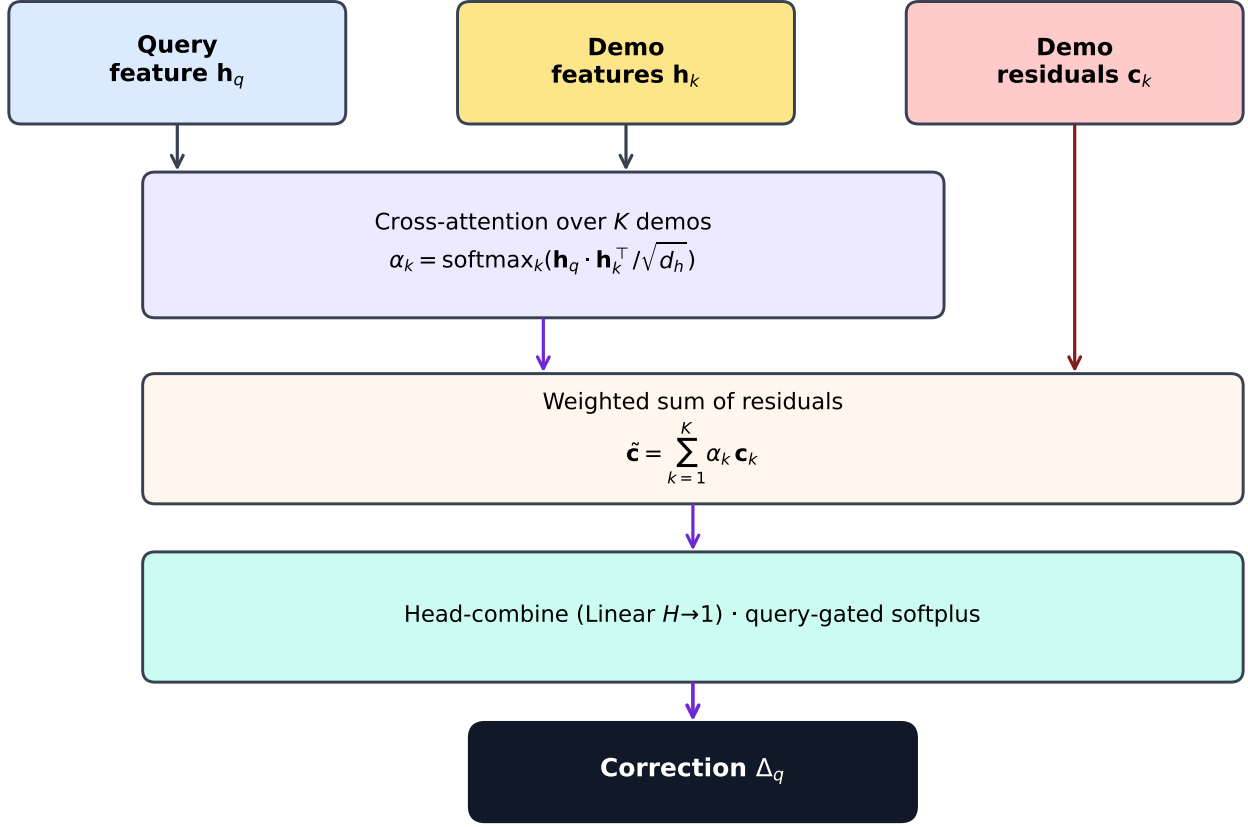


Fig. 3. Cross-attention aggregator: softmax attention between query and demo features (Q, K) weights cached demo residuals (V), then a head-combine with a query-gated softplus produces the correction  $\Delta_q$ .

**Base model.** OpenCity-plus [3] with embedding dimension 512, encoder depth 6, 16 attention heads, and  $\sim 26\text{M}$  parameters. For DIDI datasets we use input/output window = 144 steps (288 / (10/5)) with PatchEmbedding gap = 2, as prescribed by the OpenCity recipe.

**Metrics.** Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE, %), and Pearson correlation (CORR), computed with the standard masked evaluation (mae threshold = 0, mape threshold = 0.001).

**Implementation.** PyTorch 2.4.1 on NVIDIA L40S (46 GB), fp32 precision throughout.

#### B. Main Results on PEMS07M

Table I summarises our main PEMS07M results, from which four observations stand out. First, random demonstrations hurt badly (MAE 4.50  $\rightarrow$  6.14,  $-36\%$ ) because uncorrelated corrections add noise. Second, raw-input KNN recovers most of that damage (6.14  $\rightarrow$  4.66) but still does not surpass zero-shot, so retrieval alone is necessary but not sufficient. Third, learned aggregation surpasses zero-shot, with SIMPLEDEMOAGGREGATOR reaching 4.47 and DEMOAGGREGATOR reaching 4.30.

TABLE I  
MAIN RESULTS ON PEMS07M. ALL ICL METHODS USE  $K=3$ . BEST IN **BOLD**. DASHES FOR FULL-SHOT BASELINES INDICATE METRICS NOT REPORTED IN [3].

| Method                                                     | Retrieval      | Params | MAE $\downarrow$ | RMSE $\downarrow$ | MAPE% $\downarrow$ | CORR $\uparrow$ |
|------------------------------------------------------------|----------------|--------|------------------|-------------------|--------------------|-----------------|
| Zero-Shot                                                  | —              | 0      | 4.50             | 8.21              | 12.20              | 0.736           |
| ICL-Traffic / Random                                       | random         | 0      | 6.14             | 9.53              | 14.84              | 0.634           |
| ICL-Traffic / Naive avg                                    | raw-L2         | 0      | 4.66             | 8.11              | 11.92              | 0.729           |
| ICL-Traffic / Simple agg                                   | raw-L2         | 8K     | 4.47             | 8.01              | 11.55              | 0.737           |
| ICL-Traffic / Cross-attn (3ep)                             | raw-L2         | 198K   | 4.30             | 7.68              | 11.37              | 0.748           |
| <b>ICL-Traffic / Cross-attn (5ep)</b>                      | <b>emb-KNN</b> | 198K   | <b>4.15</b>      | <b>7.61</b>       | <b>11.25</b>       | <b>0.739</b>    |
| <i>Full-shot supervised baselines (as reported by [3])</i> |                |        |                  |                   |                    |                 |
| GWN [2]                                                    | —              | full   | 4.17             | —                 | —                  | —               |
| ASTGCN [10]                                                | —              | full   | 4.39             | —                 | —                  | —               |
| STGCN [1]                                                  | —              | full   | 4.44             | —                 | —                  | —               |

Fourth, swapping raw-L2 for embedding-space KNN under the same learned aggregator delivers the best result (MAE 4.15, RMSE 7.61, MAPE 11.25%, CORR 0.739), a  $+7.8\%$  MAE gain over zero-shot with only 0.8% extra parameters and no modification to the backbone.

TABLE II

CROSS-DATASET ADAPTATION. ICL AGGREGATOR AND FINE-TUNE BOTH USE 3 EPOCHS ON TARGET-DOMAIN TRAINING DATA. BEST IN **BOLD**, BEST FROZEN-BACKBONE IN UNDERLINE.

| Dataset | Method               | Backbone | MAE↓        | RMSE↓       | MAPE%↓       | CORR↑        |
|---------|----------------------|----------|-------------|-------------|--------------|--------------|
| SZ-DIDI | Zero-Shot            | frozen   | 4.66        | 7.10        | 19.05        | 0.322        |
|         | ICL-Traffic          | frozen   | <u>2.70</u> | <u>4.11</u> | <u>11.08</u> | <u>0.540</u> |
|         | Fine-tune head (3ep) | modified | <b>2.33</b> | <b>3.75</b> | <b>9.89</b>  | <b>0.681</b> |
| CD-DIDI | Zero-Shot            | frozen   | 6.34        | 9.36        | 28.36        | 0.336        |
|         | ICL-Traffic          | frozen   | <u>3.41</u> | <u>5.02</u> | <u>15.28</u> | <u>0.720</u> |
|         | Fine-tune head (3ep) | modified | <b>2.60</b> | <b>3.93</b> | <b>11.98</b> | <b>0.793</b> |

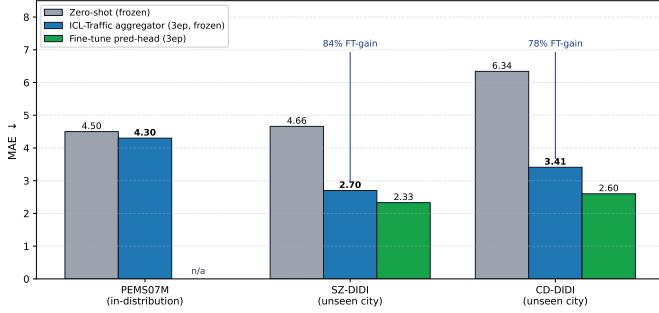


Fig. 4. Cross-dataset adaptation on SZ-DIDI and CD-DIDI (3-epoch budget): zero-shot, frozen-backbone ICL-Traffic aggregator, and prediction-head fine-tuning. FT-gain recovery is annotated above each aggregator bar.

### C. Cross-Dataset Generalization: ICL vs. Fine-Tuning

A more important question is whether output-space ICL can adapt to genuinely out-of-distribution cities and how it compares to gradient-based fine-tuning at the same budget. Table II and Fig. 4 report this comparison on SZ-DIDI and CD-DIDI. The two adaptation methods (our 3-epoch cross-attention aggregator and OpenCity’s 3-epoch prediction-head fine-tuning) use the same target-domain training set and the same 3-epoch budget, with zero-shot as the no-adaptation reference. The only difference between the two is how the training data is used.

ICL’s improvement scales with distribution shift. On in-distribution PEMS07M the aggregator improves MAE by 4.4–7.8%, whereas on SZ-DIDI the gain is 42% and on CD-DIDI it is 46%. Under strong shift the backbone makes structured errors such as chronic under-prediction of demand peaks, and KNN-retrieved demos expose exactly those biases, giving the aggregator a high signal-to-noise correction.

We quantify how much of fine-tuning’s gain is reachable from output space alone. Define

$$\text{Recovery} = \frac{\text{MAE}_{\text{ZS}} - \text{MAE}_{\text{ICL}}}{\text{MAE}_{\text{ZS}} - \text{MAE}_{\text{FT}}}. \quad (13)$$

ICL recovers 84% on SZ-DIDI and 78% on CD-DIDI with a fully frozen backbone. The remaining ~20% requires updating the prediction-head weights to map encoder features to the new output distribution, which is unreachable by any output-space correction by construction.

TABLE III

ABLATION: DEMONSTRATION SELECTION STRATEGY. ALL WITH NAIVE AVERAGING.  $S$ : SAMPLING ROUNDS.

| Selection    | $K$ | $S$ | MAE  | RMSE  | vs. ZS |
|--------------|-----|-----|------|-------|--------|
| Random       | 1   | 1   | 7.39 | 12.00 | −64%   |
| Random       | 3   | 1   | 6.14 | 9.53  | −36%   |
| Random       | 1   | 10  | 5.24 | 8.52  | −16%   |
| KNN (raw-L2) | 1   | 1   | 5.61 | 10.04 | −25%   |
| KNN (raw-L2) | 3   | 1   | 4.66 | 8.11  | −3.6%  |

TABLE IV

ABLATION: AGGREGATION ARCHITECTURE ON PEMS07M. ALL WITH  $K=3$ .

| Aggregator              | Retrieval      | Params | MAE↓        | RMSE↓       | MAPE%↓       | CORR↑        |
|-------------------------|----------------|--------|-------------|-------------|--------------|--------------|
| Naive avg               | raw-L2         | 0      | 4.66        | 8.11        | 11.92        | 0.729        |
| Cosine (5ep)            | raw-L2         | 8K     | 4.47        | 8.01        | 11.55        | 0.737        |
| Cross-attn (3ep)        | raw-L2         | 198K   | 4.30        | 7.68        | 11.37        | 0.748        |
| Cross-attn (10ep)       | raw-L2         | 198K   | 4.29        | 7.71        | 11.42        | 0.748        |
| <b>Cross-attn (5ep)</b> | <b>emb-KNN</b> | 198K   | <b>4.15</b> | <b>7.61</b> | <b>11.25</b> | <b>0.739</b> |

### D. Ablation: Demonstration Selection Strategy

Table III isolates the effect of demonstration retrieval with naive averaging. Random selection with  $K=1$  yields MAE 7.39 (−64% vs. zero-shot). Increasing to  $K=3$  reduces variance but remains far below zero-shot (−36%). Multi-round sampling ( $S=10$ ) with random selection partially recovers by averaging out noise (−16%) but cannot match KNN. With KNN, even  $K=1$  (MAE 5.61) outperforms random  $K=3$ , and  $K=3$  (MAE 4.66) approaches zero-shot. Demo-query alignment is the dominant factor.

### E. Ablation: Aggregation Architecture

Table IV compares aggregation strategies at  $K=3$ . Naive averaging (0 parameters) achieves MAE 4.66 but cannot surpass zero-shot. SIMPLEDEMOAGGREGATOR (8K) learns cosine-similarity weights and surpasses zero-shot (4.47). DEMOAGGREGATOR (198K) captures richer query-demo interactions and reaches 4.30 at 3 epochs; continuing to 10 epochs adds only 0.01 MAE, confirming rapid convergence from cached features. Replacing raw-L2 with embedding-KNN under the same 198K aggregator delivers the overall best result of 4.15 in 5 epochs.

### F. Analysis

*Why random demonstrations hurt:* Random corrections are uncorrelated with the query’s prediction error, so averaging them just adds noise. As  $S \rightarrow \infty$ , the average correction converges to the population mean residual, which Phase 1 statistics confirm is nearly zero, so random-demo ICL recovers zero-shot at best and never surpasses it.

*Why KNN is necessary but not sufficient:* Similar inputs yield correlated prediction errors, so KNN-retrieved corrections carry real signal. Naive uniform averaging, however, dilutes this signal because even among the  $K$  nearest neighbors, residual noise remains, and nodes that match the query well

average with nodes that match poorly. A learned weighting is needed to focus on the most relevant corrections and to do so *per node*.

*Why learned aggregation surpasses zero-shot:* The aggregator learns per-node, per-head weights that up-weight the most informative demonstrations. Even the 8K-parameter simple aggregator is enough to beat zero-shot, confirming that the missing ingredient on top of KNN is weighting rather than any richer transformation. The 198K cross-attention variant then captures richer interactions through its multi-head structure and query-conditioned gate, yielding the largest further gain.

*Convergence:* The aggregator converges by epoch 3–4 from cached features. The 3-epoch row (MAE 4.30 on PEMS07M) is deliberately aligned with OpenCity’s 3-epoch fast-adaptation protocol so the cross-dataset comparison (Section IV-C) is budget-matched; extending to 5 epochs only matters when retrieval quality has improved (embedding KNN).

*Failed approach: naive sequence extension:* We initially attempted standard ICL by concatenating demonstrations into the model’s input sequence. This caused out-of-distribution self-attention patterns and positional encoding mismatches, degrading MAE by 9–28% versus zero-shot. This failure motivated our output-space approach.

*Per-query retrieval vs. fine-tuning:* Fine-tuning treats all training examples equally and produces a single globally adapted model. Our method selects the few most similar examples per query and weights their corrections accordingly, preserving local structure that a global update would average out. This explains why ICL’s relative gain over zero-shot grows with the size of the distribution shift.

## V. DISCUSSION AND FUTURE WORK

### A. Limitations

*Compute overhead:* Our method nominally requires  $K+1$  forward passes per test sample ( $4\times$  for  $K=3$ ). In deployment, all demonstration passes are precomputed once in Phase 1. The marginal test-time cost is then the single query pass plus a KNN lookup plus a 198K-parameter aggregator call, which is negligible.

*Output-space correction, not true ICL:* Demonstrations never enter the model’s attention context, and corrections are applied in output space only. This is a pragmatic constraint of the fixed-length pretraining of current ST-FMs. True ICL, where the model attends jointly over demonstrations and query as in ICT [5] and TimesFM-ICF [7], would require architectural modification and continued pretraining.

*Retrieval ceiling:* Raw-input L2 matches by point-wise numerical closeness, not by traffic-pattern similarity. Embedding-space KNN is better because the encoder captures task-relevant structure, but the encoder was not trained for retrieval, so nearby demos in feature space are not necessarily the ones whose corrections help most. We discuss directions for closing this gap in Section V-B.

*Trainable parameters:* Our aggregator requires an offline training phase from cached features. Like fine-tuning, we use target-domain training data. The difference is in *how* that data is used: fine-tuning absorbs it into model weights; ICL-Traffic preserves it as a retrievable demo pool and the aggregator learns only a general weighting strategy. At inference, per-query variation comes from demo retrieval, not from aggregator weights (which are fixed).

*Capability ceiling:* Output-space correction recovers  $\sim 80\%$  of head fine-tuning’s MAE gain on strongly OOD cities. The remaining 20% requires updating the prediction head, which output-space methods cannot reach. A natural next step is to apply ICL first and then fine-tune only the head on the residual error that ICL leaves behind.

### B. Future Work

*Better retrieval:* Several directions can narrow the retrieval ceiling, including training a task-aware retrieval metric that optimises for correction usefulness, incorporating temporal priors, and exploring distance functions beyond L2.

*Richer aggregation:* The aggregator could learn horizon- or graph-aware weights, for example a GCN layer that smooths per-node attention over the road network.

## VI. CONCLUSION

We have presented ICL-Traffic, a lightweight in-context learning framework for spatio-temporal foundation models that keeps the backbone entirely frozen. Our experiments show that demonstration retrieval and a learned aggregator are both necessary and jointly sufficient to act as an *output-space surrogate* for gradient-based adaptation. Across in-distribution and cross-city settings, this surrogate recovers up to 84% of the MAE gain that prediction-head fine-tuning achieves, with only 0.8% additional parameters and no test-time gradients. The remaining gap is structural, arising from the inability of output-space corrections to remap encoder features to a new output distribution.

## REFERENCES

- [1] B. Yu, H. Yin, and Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2018.
- [2] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, “Graph WaveNet for deep spatial-temporal graph modeling,” in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2019.
- [3] Z. Li, L. Xia, L. Shi, Y. Xu, D. Yin, and C. Huang, “OpenCity: Open spatio-temporal foundation models for traffic prediction,” in *Proceedings of ACM Conference*, 2024.
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [5] Y. Chen, R. Zhong, S. Zha, G. Karypis, and H. He, “Meta-learning via language model in-context tuning,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- [6] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *Proceedings of the 34th International Conference on Machine Learning*, pp. 1126–1135, 2017.
- [7] A. Das, M. Faw, R. Sen, and Y. Zhou, “In-context fine-tuning for time-series foundation models,” *arXiv preprint arXiv:2410.24087*, 2024.

- [8] A. Das, W. Kong, R. Sen, and Y. Zhou, “A decoder-only foundation model for time-series forecasting,” in *International Conference on Machine Learning*, 2024.
- [9] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, “Adaptive graph convolutional recurrent network for traffic forecasting,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 17804–17815, 2020.
- [10] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, “Attention based spatial-temporal graph convolutional networks for traffic flow forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 922–929, 2019.
- [11] Y. Yuan, J. Ding, J. Feng, D. Jin, and Y. Li, “UniST: A prompt-empowered universal model for urban spatio-temporal prediction,” *arXiv preprint arXiv:2402.11838*, 2024.
- [12] Z. Li, L. Xia, J. Tang, Y. Xu, L. Shi, L. Xia, D. Yin, and C. Huang, “UrbanGPT: Spatio-temporal large language models,” *arXiv preprint arXiv:2403.00813*, 2024.
- [13] Z. Li, L. Xia, Y. Xu, and C. Huang, “FlashST: A simple and universal prompt-tuning framework for traffic prediction,” in *International Conference on Machine Learning*, pp. 28978–28988, 2024.
- [14] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, “Tent: Fully test-time adaptation by entropy minimization,” in *International Conference on Learning Representations*, 2021.
- [15] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt, “Test-time training with self-supervision for generalization under distribution shifts,” in *International Conference on Machine Learning*, pp. 9229–9248, 2020.